



Chapter 2

Object-Oriented Programming (OOP) in Dart

2.1 Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects rather than functions or logic.

An object represents a real-world entity — it has attributes (data) and methods (behavior) that operate on that data.

Dart, being a fully object-oriented language, treats everything as an object, even primitive data types like numbers, strings, and booleans.

This makes OOP not just a feature but the core foundation of Dart and Flutter development

2.2 The Four Pillars of OOP

The four main principles of Object-Oriented Programming are:

- Encapsulation: Hiding internal data and exposing only necessary functionality.
- Abstraction: Simplifying complex systems by modeling only essential features.
- Inheritance: Allowing new classes to reuse properties and methods of existing ones.
- Polymorphism: Allowing objects to take many forms, usually through method overriding.

Dart also adds two additional advanced OOP concepts:

- **Mixins** (for modular code reuse)
- **Generics** (for type-safe flexibility)

2.3 Class and Object



A **class** is a **blueprint or template** that defines how an object should look and behave.

An **object** is an instance created from that blueprint.

Think of a class as a “design” and an object as the “real thing” built from it

Example:

```
class Human {  
    int numberOfArms;  
    double height;  
    double weight;  
    String hairColor;  
  
    void read() {  
        print("This human is reading.");  
    }  
}  
  
void main() {  
    Human jon = Human();  
    jon.numberOfArms = 2;  
    jon.height = 1.75;  
    jon.weight = 70;  
    jon.hairColor = "Black";  
  
    jon.read();  
}
```

- The **class Human** defines attributes (fields) and behaviors (methods).
- The **object jon** is an instance that stores its own data

Difference between Function and Method

Function	Method
Stand-alone piece of code, not tied to any class.	Belongs to a class or object.
Called directly.	Called using an object or class.



Example: <code>print("Hello");</code>	Example: <code>jon.read();</code>
---------------------------------------	-----------------------------------

2.3.1 Anonymous Objects

- In Dart (and in OOP generally), an **anonymous object** is an **object that is created without assigning it to a variable name**.
- It is often used when the object is needed **only once**, typically for **temporary operations** or **immediate method calls**.

Anonymous objects are extremely useful when:

- You don't need to reuse the object later.
- You just want to invoke a method quickly on a new instance

Simple Anonymous Object:

```
class Car {  
  void start() {  
    print("Car started");  
  }  
}  
  
void main() {  
  // Creating and using an anonymous object  
  Car().start();  
}
```

- Here, `Car()` creates an object **without a name**.
- The method `.start()` is called immediately on that unnamed instance.
- The object is automatically destroyed after the line executes, since no variable keeps a reference to it

2.4 Encapsulation

Encapsulation is the process of bundling data (attributes) and methods (functions) that operate on that data into a single unit (class), while controlling how that data is accessed or modified.

This ensures that data is protected from unintended modification — known as data hiding.



In Dart, encapsulation is achieved using:

Private members: Prefixing variable or method names with `_` makes them private to their library.

Getters and Setters: Used to control access to private properties

Example of Encapsulation

```
class BankAccount {
  double _balance = 0.0; // Private variable

  void deposit(double amount) {
    if (amount > 0) {
      _balance += amount;
    }
  }

  void withdraw(double amount) {
    if (amount <= _balance) {
      _balance -= amount;
    } else {
      print("Insufficient funds!");
    }
  }

  double get balance => _balance; // Getter
}

void main() {
  var account = BankAccount();
  account.deposit(1000);
  account.withdraw(250);
  print("Current balance: ${account.balance}");
}
```

- The `_balance` variable is private, meaning it cannot be accessed directly outside the class. The user can only interact with it through methods (deposit, withdraw, and the getter)

2.5 Abstraction

Abstraction is the process of **hiding complex implementation details** and showing only the necessary features of an object. It simplifies code and makes it easier to maintain and extend.

In Dart, abstraction is achieved using **abstract classes** and **interfaces**.

Example :

```
abstract class Vehicle {  
    void start(); // abstract method  
    void stop(); // abstract method  
}  
  
class Car extends Vehicle {  
    @override  
    void start() {  
        print("Car started");  
    }  
  
    @override  
    void stop() {  
        print("Car stopped");  
    }  
}  
  
void main() {  
    Vehicle myCar = Car();  
    myCar.start();  
    myCar.stop();  
}
```

- Vehicle is an abstract class — it **cannot be instantiated directly**.
- The Car class **implements** the abstract methods with specific details

2.6 Inheritance

- Inheritance allows one class to **inherit** properties and methods from another.
- This supports **code reuse** and a **hierarchical relationship** between classes

Example of Inheritance:

```
class Animal {
    void eat() {
        print("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        print("Dog is barking");
    }
}

void main() {
    Dog dog = Dog();
    dog.eat(); // Inherited method
    dog.bark(); // Child method
}
```

- Dog inherits eat() from Animal.
- This avoids rewriting the same behavior multiple times

2.7 Polymorphism

- Polymorphism (meaning “many forms”) allows objects of different types to be treated as objects of a common superclass.



- It enables **method overriding** — redefining inherited methods to perform specific actions.

Example of polymorphism:

```
class Animal {
    void makeSound() {
        print("Animal makes a sound");
    }
}

class Cat extends Animal {
    @override
    void makeSound() {
        print("Meow");
    }
}

class Dog extends Animal {
    @override
    void makeSound() {
        print("Woof");
    }
}

void main() {
    List<Animal> animals = [Cat(), Dog()];
    for (var a in animals) {
        a.makeSound(); // Calls the appropriate overridden method
    }
}
```

2.8 Mixins

- A **mixin** is a way to **reuse code across multiple classes** without using inheritance.
- A mixin is like a partial class that can be “mixed in” to another class



```

mixin Swimmer {
  void swim() => print("Swimming...");
}

mixin Flyer {
  void fly() => print("Flying...");
}

class Duck with Swimmer, Flyer {
  void showAbilities() {
    swim();
    fly();
  }
}

void main() {
  Duck d = Duck();
  d.showAbilities();
}

```

- The Duck class combines two behaviors (swim + fly) using mixins.
- This avoids complex inheritance chains.

2.9 Enumeration (Enums):

- An **enum** defines a **set of named constant values**.
- It's useful when a variable can only take one of a few predefined options.



```
enum Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }

void main() {
    Day today = Day.Friday;

    switch (today) {
        case Day.Friday:
            print("Weekend is near!");
            break;
        default:
            print("Working day.");
    }
}
```

2.10 Generics

- Generics allow you to create **classes and methods** that work with **different data types** while maintaining **type safety**.
- They make code reusable and prevent runtime type errors

Example of Generics:



```
class Box<T> {  
    T content;  
  
    Box(this.content);  
  
    void showContent() {  
        print("Content: $content");  
    }  
}  
  
void main() {  
    var intBox = Box<int>(100);  
    var strBox = Box<String>("Hello Dart");  
  
    intBox.showContent();  
    strBox.showContent();  
}
```